Software Project Management – IS212

Visualising Software Architecture & Code

Announcements

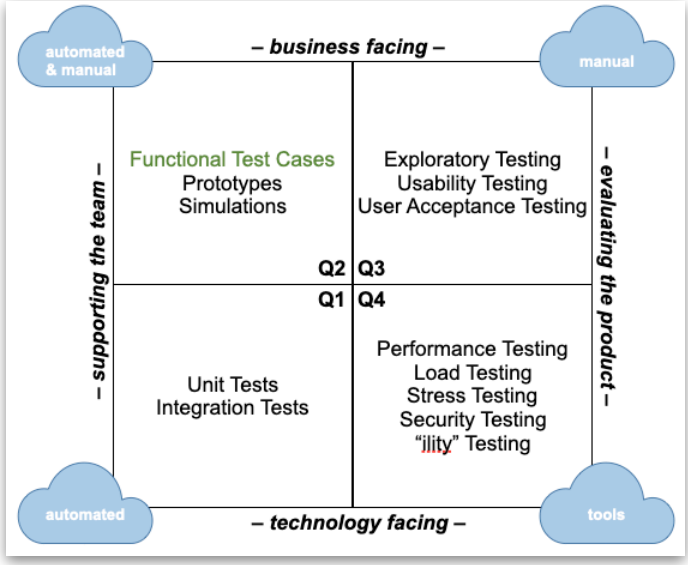
Announcement – *Start Sprinting!* 🏁

- If you haven't started sprinting yet, please do so **this week**
- If you don't, and you're aiming for three 2-week sprints, then you will have limited buffer time before the Friday Week 12 due date
- The classes in Weeks 1–4 have covered the essential content on running a scrum process
- The remaining classes will help you to improve your process and deliver higher-quality software
  - *Incorporate the content from the later classes into your later sprint(s)*
  - *Don't worry about being 'perfect' from the start* 😊

Recap

Week 4 Recap – Agile Testing & Test Cases

- How does testing fit into an agile software development process?
- Which tests to cover in the agile testing quadrant?
- How do we derive test cases from ACs?
- What's a happy, negative, and boundary test?
- When do we write tests in scrum?
- Why are regression testing and test traceability important?



Week 4 Recap – Git

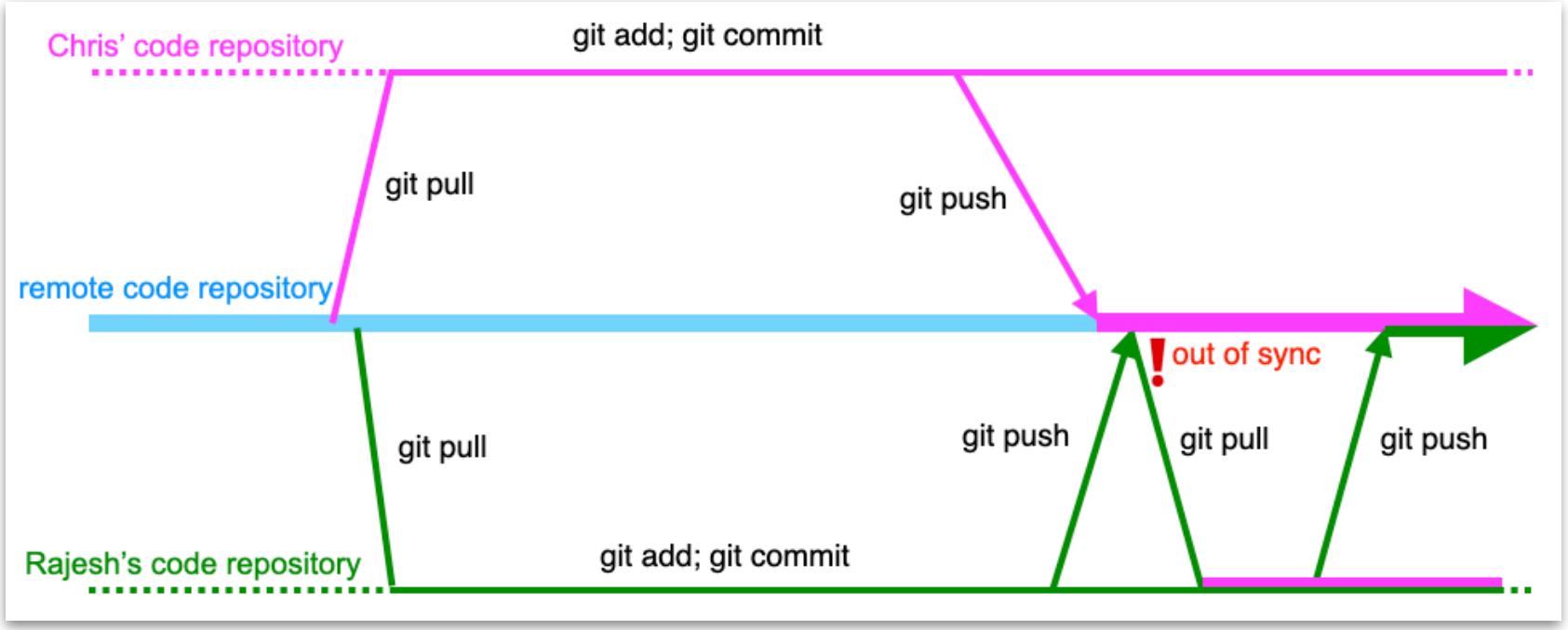
git clone

git pull

git add

git commit

git push



Today: Visualising Software Architecture & Code

Objectives

On completing this class, you will be able to:

- Explain the rationale for visualising software architecture
- Model the context, containers, and components of a system using the C4 notation
- Model object-oriented relationships in code using UML class diagrams

Outline

- ***Visualising Architecture*** **–** Software architecture, C4 model, examples
- ***Visualising Code*** **–** UML class diagrams, abstraction, encapsulation, association, inheritance, polymorphism

SMU Classification: Restricted

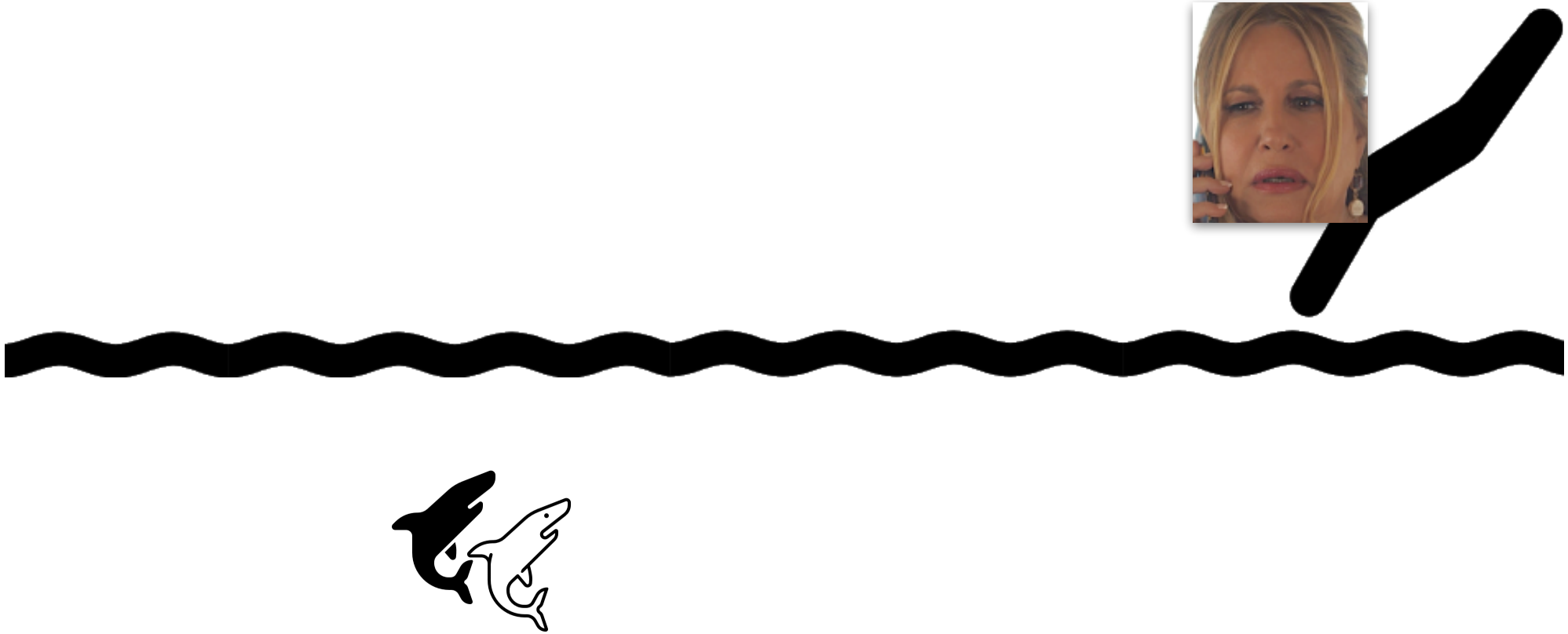
Software Architecture

User Story
"As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently."
Acceptance Criteria Ensure that the customer: • Can input a valid card number, expiration date, CVV, and cardholder name • Can only pay with Visa, Mastercard, or American Express • Is requested to provide an OTP • Is immediately informed of payment success or failure • Has a secure SSL/TLS connection

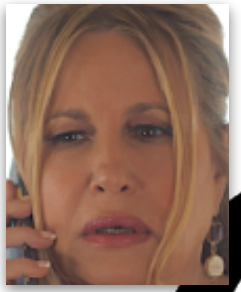


Developer

Diving into coding… without a blueprint

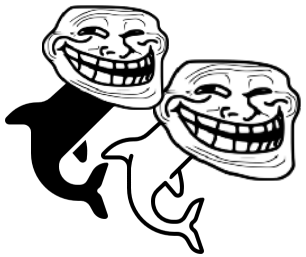


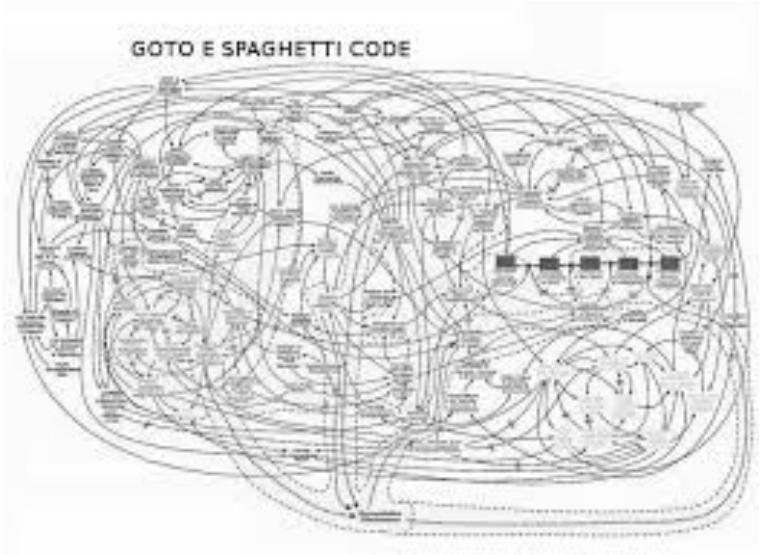
Diving into coding… without a blueprint





“big ball of mud”





Software Architecture

- Software architecture is the high-level structure of the system
  - *Defines the major components, their relationships, and interactions*
  - *The principles guiding their design and evolution*

- Like a "blueprint" for the system and project
  - *Builds a shared understanding for the team*
  - *Ensures the team understands the context of the components they build*

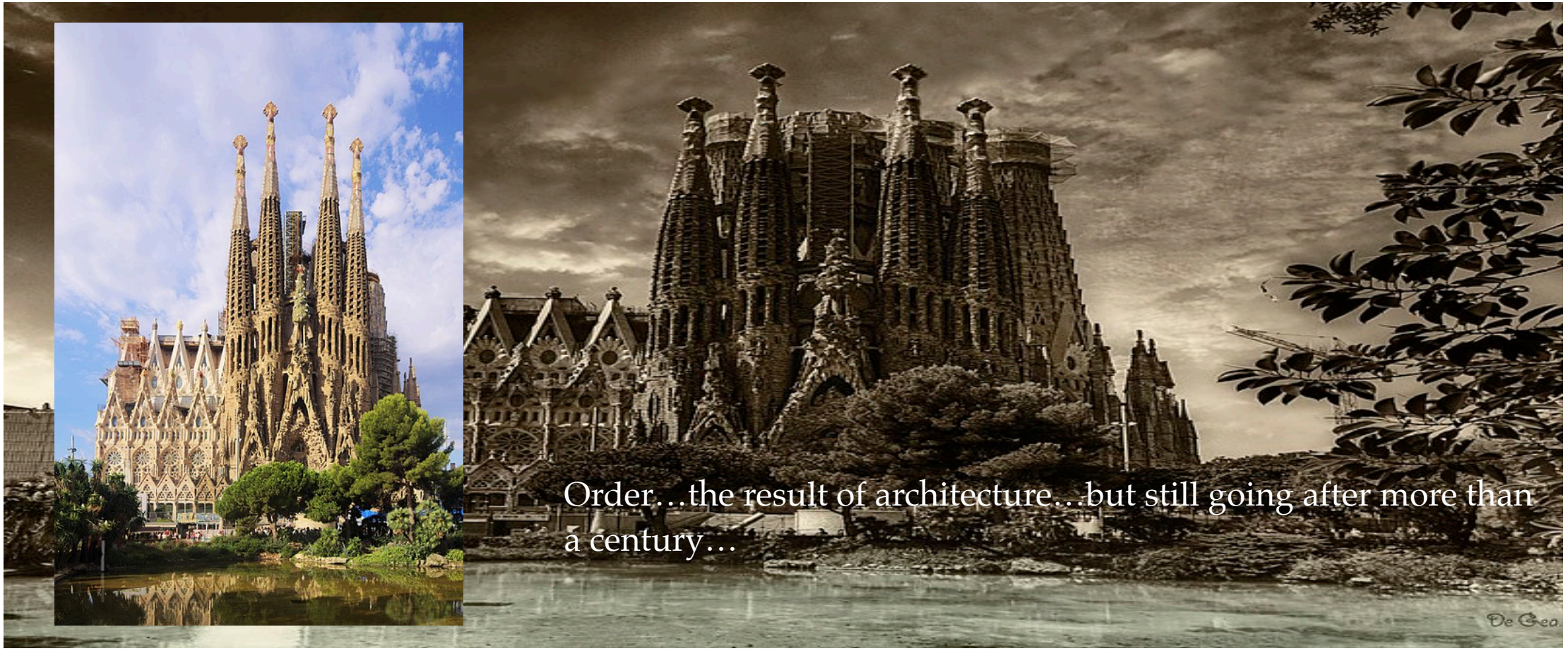
- Agile projects favour code over documentation, but this doesn't mean that there should be no documentation
  - *We need **"just enough"** software architecture design*
  - *i.e. architectural models that are lightweight and accessible to different stakeholders*

No Architecture in Mind



Winchester Mystery House by Www78. Licensed under Creative Commons Attribution-Share Alike 3.0 Unported.

Too Much Architecture in Mind



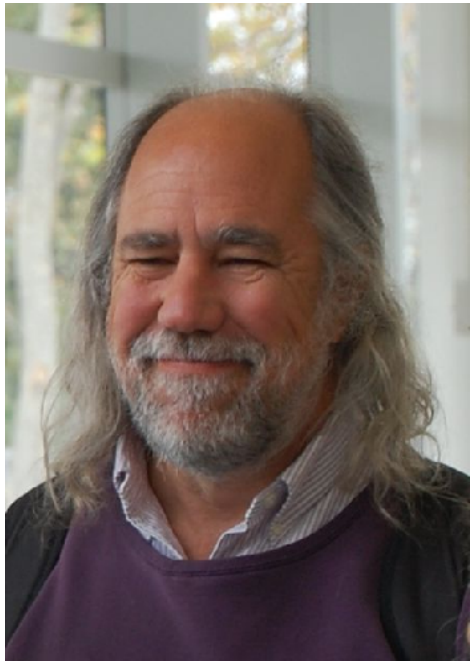
Basilica de la Sagrada Familia by Jose Ramirez. Licensed under Creative Commons Attribution-Share Alike 2.0 Generic.

Architecture Represents **Significant** Design Decisions

All architecture is design but not all design is architecture.

Architecture represents the significant design decisions that shape a system, where significant is measured by cost of change.

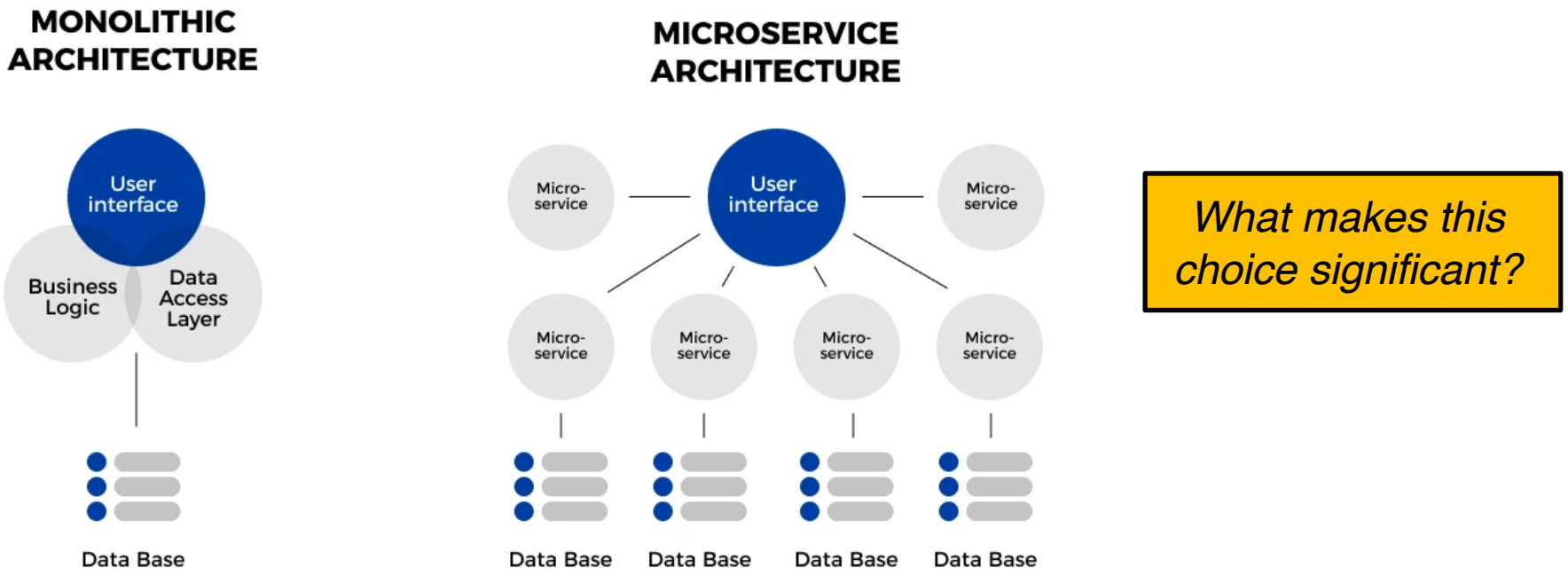
– Grady Booch, *On Design, Handbook of Software Architecture blog, March 2006*



Examples of Significant Design Decisions

- Recall the software architectural patterns you encountered in other courses – these are good examples of **significant** design choices

- For examples, monolithic vs. microservices architectures:



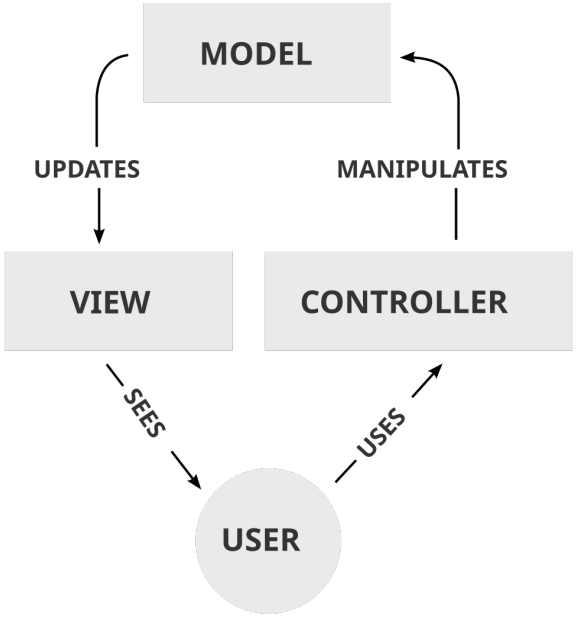
What makes this choice significant?

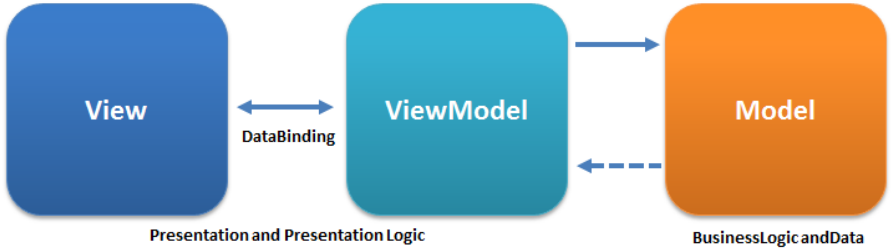
Image Source: https://www.openlegacy.com/blog/monolithic-application

Examples of Significant Design Decisions

- You have also encountered architectural patterns regarding the separation of GUIs from business and backend logic

- For example, Model-View-Controller vs. Model-View-ViewModel





Vue.js

Agile projects favour code over documentation, but this doesn't mean that there should be no documentation

- *We need **"just enough"** software architecture design*
- *i.e. architectural models that are lightweight and accessible to different stakeholders*

C4 Model

a lean graphical modelling approach
for software architecture

C4 Model – *an "abstraction-first" approach*



Visualising software architecture with the C4 model - Simon Brown, Agile on the Beach 2019

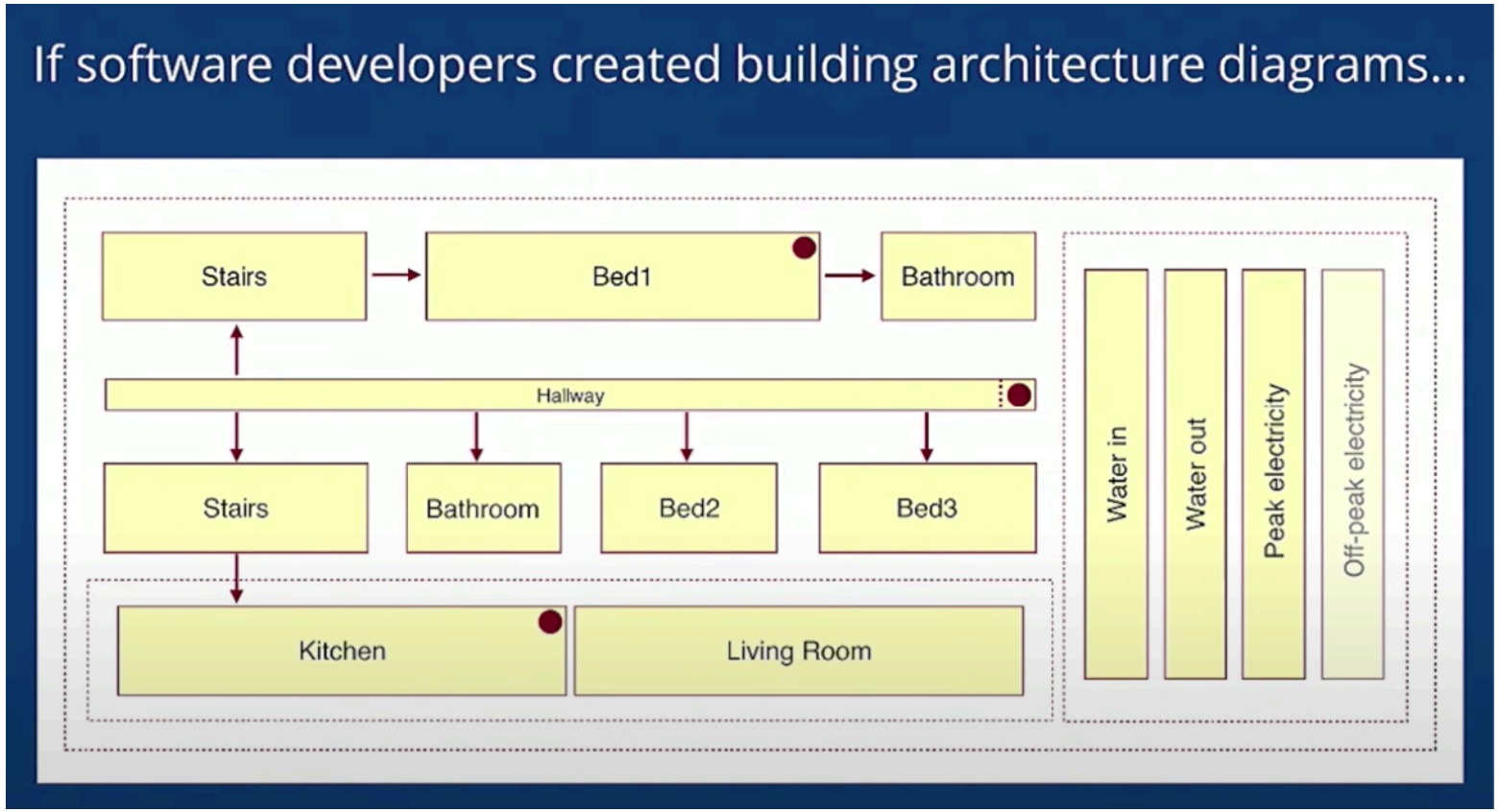
"Abstraction over notation"

what exactly
is **abstraction**?

https://www.youtube.com/watch?v=x2-rSnhpw0g

Aside – What is Abstraction?

- This is abstract… but "arbitrary"
  - *The "yellow box" notation appears to mean many things…*



Credit: Simon Brown, Agile on the Beach

Aside – What is Abstraction?

"The purpose of abstracting is not to be vague…

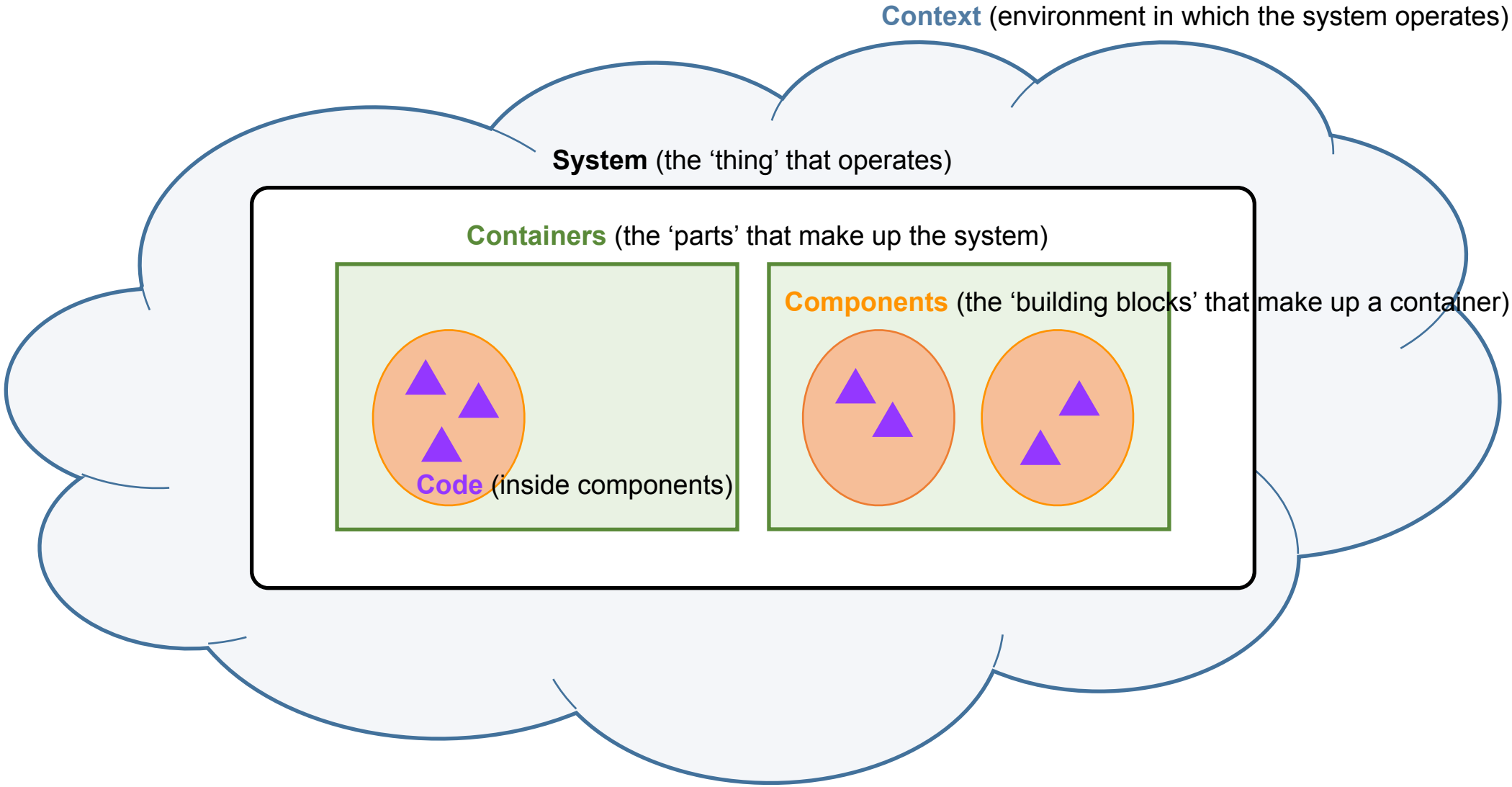
…but to create a new semantic level in which one can be absolutely precise."

What do we want to be precise about in software architecture?



Edsger Dijkstra

C4 Model *(**C**ontext, **C**ontainers, **C**omponents, **C**ode)*



Concept: Simon Brown, *C4 (context, container, component, class) approach to visualising software*

C4 Model *(**C**ontext, **C**ontainers, **C**omponents, **C**ode)*

- The C4 model is a framework for visualising the architecture of a software system via a hierarchical set of diagrams
  - *Each diagram is at a **different abstraction level**, and is intended for different stakeholders*
  - *C4 notation is supported by* draw.io *(search for 'C4' in the shapes catalogue)*

- **C1 (Context) –** high-level overview of the environment the system operates in (e.g. users, other systems, external services)

- **C2 (Containers) –** what are the major containers (applications, services, databases, …) that make up the system

- **C3 (Component) –** as necessary – diagram showing the internal structure of a container (e.g. the structure of a single application)

- **C4 (Code) –** as necessary – diagram visualising the implementation of a component, e.g. the classes and methods of object-oriented code

Running Example – Integrated Billing for Hospitals

- Suppose you are working for a hospital that has multiple completely separate legacy systems for different patient costs, e.g.
  - *A legacy pharmacy billing system*
  - *A legacy ward billing system*
- Your team is developing a modern web application that allows all of these costs to be retrieved and consolidated in a single place
- What might the software architecture look like in C4?



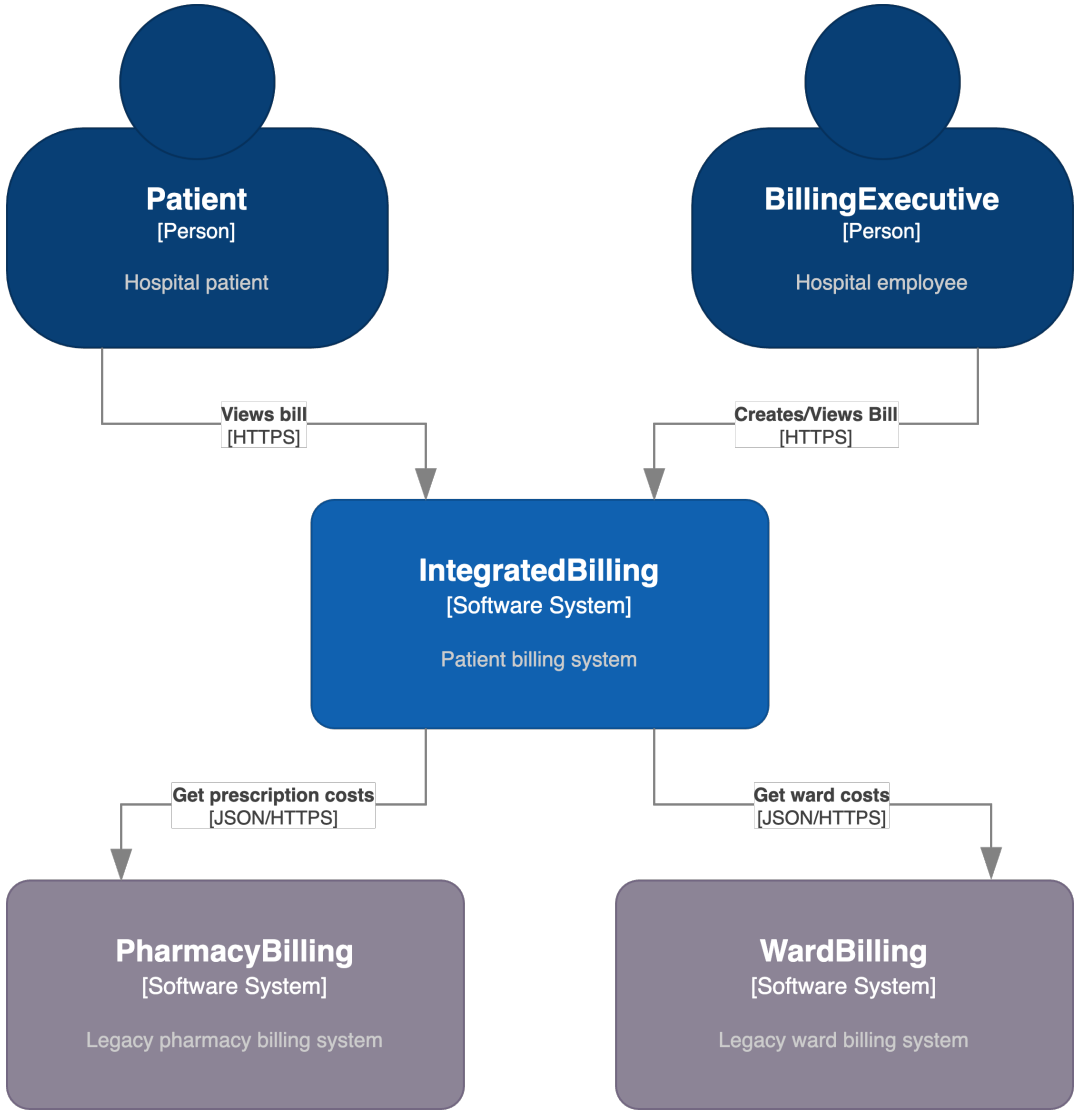
Source: https://www.healthcareittoday.com/2018/05/04/medical-billing-cartoons-fun-friday/

C1 (Context) Diagrams

- Your system is a **single box**
  - "Zoom out" and see the big picture
  - Focus is on the users and (other) systems that it interacts with

- High level of abstraction; accessible for non-technical team members

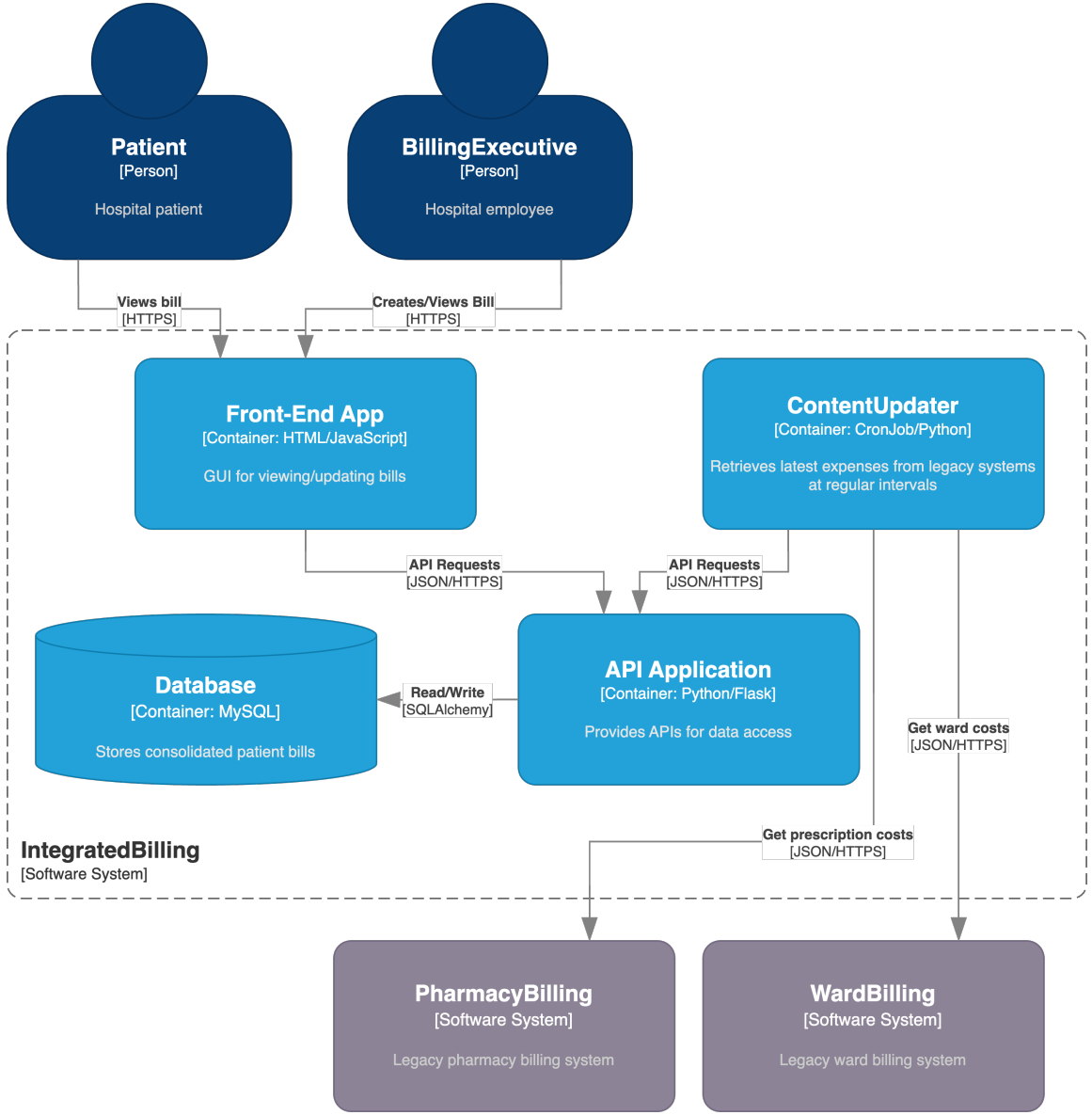
Scope: A single software system
Primary elements: The software system in scope
Supporting elements: Users (actors, roles or personas) and systems (external dependencies) that are directly connected to the software system in scope. Typically, these other systems sit outside the scope or boundary of your own software system and you do not have responsibility or ownership of them
Intended audience: Everybody, both technical and non-technical, inside and outside of the software development team
Recommended for most teams: Yes



C2 (Container) Diagrams

- "Zoom in" to your **system** (the dotted line) and depict the containers it's made up of
- Containers are (potentially) separately running processes that execute/store data
- C2 depicts the "high level shape" of the system, and major technology choices

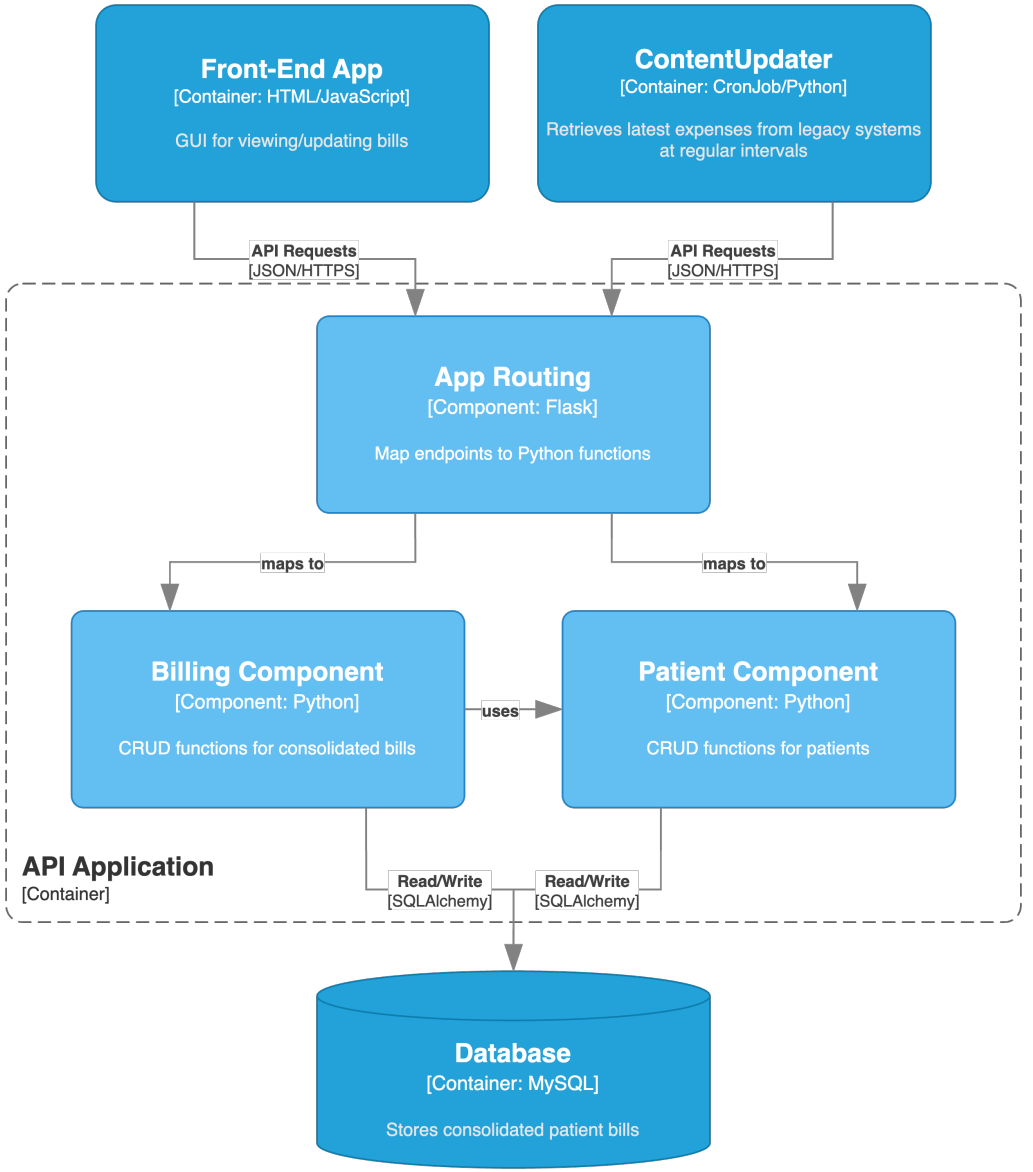
Scope: A single software system
Primary elements: Containers within the software system in scope
Supporting elements: People and software systems directly connected to the containers
Intended audience: Technical people inside and outside of the software development team, including software architects, developers, operations and support staff
Recommended for most teams: Yes



C3 (Component) Diagrams

- "Zoom in" to a **container** (the dotted line) and depict its internal structure
- The components are typically packaged together (e.g. a single Python/Flask application)
- C3 is included when necessary – only needed if it adds value (e.g. visualising the structure of a particularly complex container/app)

Scope: A single container
Primary elements: Components within the container in scope
Supporting elements: Containers (within the software system in scope), people and software systems directly connected to the components
Intended audience: Software architects and developers
Recommended for most teams: No. Only create component diagrams if you feel they add value



C4 (Code) Diagrams

- We can “zoom in” even further to visualise the structure of the code making up a specific component

- At this level, we can utilise existing notations, e.g.
  - *Unified Modelling Language (UML) for modelling code*
  - *Entity-Relationship (ER) Diagrams for modelling relational databases*

- Agile teams create such models more sparingly, e.g. focusing on particularly complex relations among object-oriented classes

- ***We will explore some*** in the second part of this class!

SMU Classification: Restricted

UML Class Diagrams for Object-Oriented Design

– a code-level graphical notation –

Visualising Code

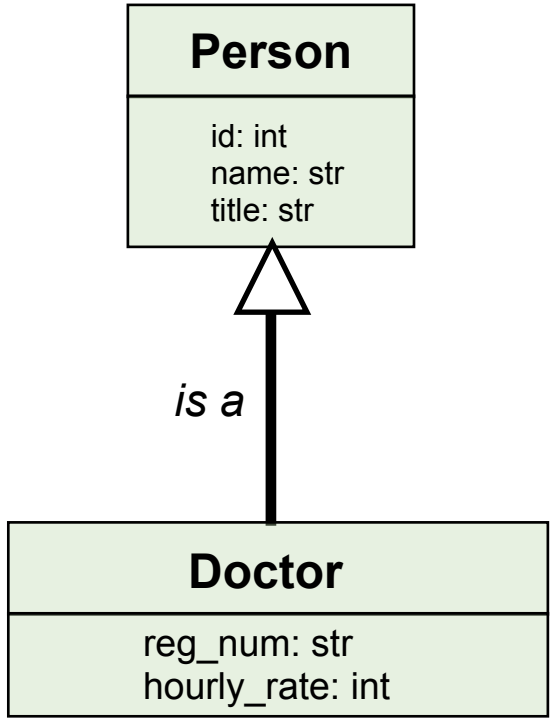
- Suppose your code is **object-oriented** (recall IS113, IS213)

```python
class Person(db.Model):
    __tablename__ = 'person'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))
    title = db.Column(db.String(10))


class Doctor(Person):
    __tablename__ = 'doctor'

    id = db.Column(db.Integer, db.ForeignKey('person.id'),
    reg_num = db.Column(db.String(15))
    hourly_rate = db.Column(db.Integer)
```



Python/Flask code

A model of the code

Unifed Modelling Language **(UML)** Notations

Category	**Diagram**	**Usage**
Structural	Class diagram	Describes the **static structure** of **object-oriented software**. The static structure is built from classes and relationships between them.
	Object diagram	Used to characterize a class diagram, usually represents the **object combinations** of a specific class diagram.
	Component diagram	Depicts the **physical organization** of software assets that would provide the needed functionality.
	Deployment diagram	Shows how the software is to **deployed in production**.
Behavioural	Use case diagram	It depicts the actors, use cases and their relationships. Use cases are **descriptions of system functionality**.
	Sequence diagram	Models the **dynamic aspects** of classes. It depicts the objects and their interactions through time-ordered messages. This is a **temporal** view.
	Collaboration diagram	Essentially the same as sequence diagram, but it emphasizes the **structural organization** of objects.
	Statechart diagram	Expresses the **lifecycle** of an object (or a system).
	Activity diagram	Is an elaborate form of flow chart diagrams. Depicts the **flow of activities** in a system.

UML for Object-Oriented Code

- We will look at (UML) class diagrams, which describe the static structure of object-oriented code

- For instance, the attributes/methods of classes, and the relationships between objects

- **In particular,** we will use class diagrams to visualise:
  - *Data abstraction and encapsulation*
  - *Association relationships*
  - *Inheritance and polymorphism*

https://www.youtube.com/watch?v=m_MQYyJpIjg

Data Abstraction

- Data abstraction is the concept of simplifying complex systems by modelling classes based on essential characteristics while hiding unnecessary details

Chris Hemsworth



abstract to…

object **a1**

name: ‘Chris Hemsworth’
date_of_birth: ‘1983-08-11’
nric: ‘HEMMY1983’

```
a1.name == 'Chris Hemsworth'
```

Abstraction is Implemented through **Classes**

*a **class** is just a "template"*

Actor

name: str
date_of_birth: str
nric: str

*an **object** is an "instance" of the template*

object **a1**

name: 'Chris Hemsworth'
date_of_birth: '1983-08-11'
nric: 'HEMMY1983'



object **a2**

name: 'Judi Dench'
date_of_birth: '1934-12-09'
nric: 'MB0ND007'



object **a3**

name: 'Maggie Smith'
date_of_birth: '1934-12-28'
nric: 'MINERVA1'

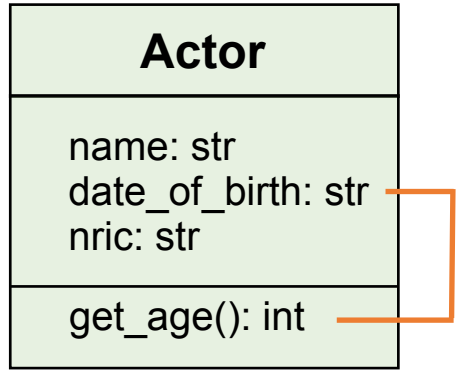


Encapsulation

- Encapsulation is the bundling of data and functions (methods) that operate on it within the same class

attributes *(data)*

methods *(functions that operate on the data)*



object **a2**

name: 'Judi Dench'
date_of_birth: '1934-12-09'
nric: 'MB0ND007'

get_age(*self*): …

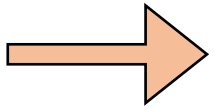


```
a2.get_age() == 90
```

Demo!

Encapsulation

- The special method **`__init__`** is (roughly) a "constructor"



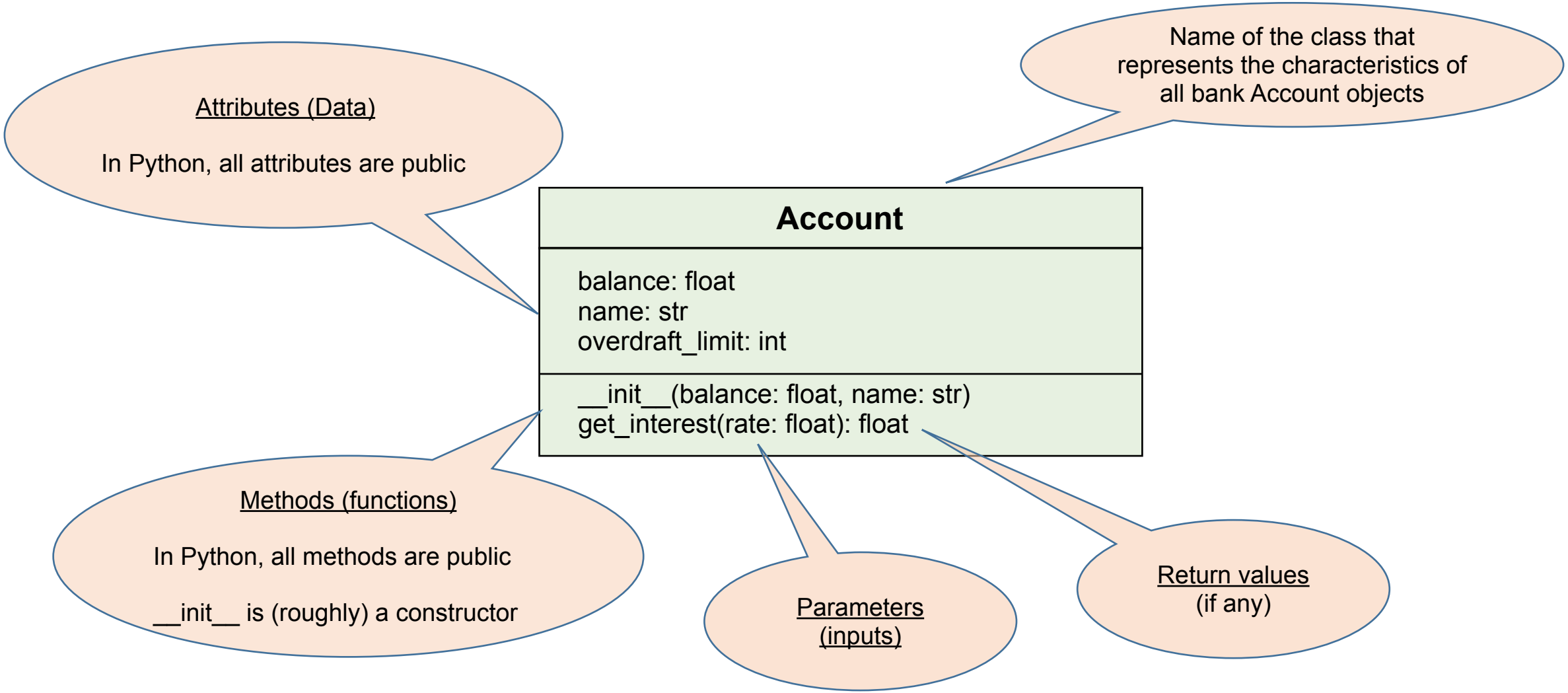
Actor
name: str date_of_birth: str nric: str
__init__(name: str, date_of_birth: str, nric: str) get_age(): int



```
a4 = Actor('Michelle Yeoh', '1962-08-06', 'LANGKAWI88')

a4.get_age() == 63
```

Example – Bank Account Class



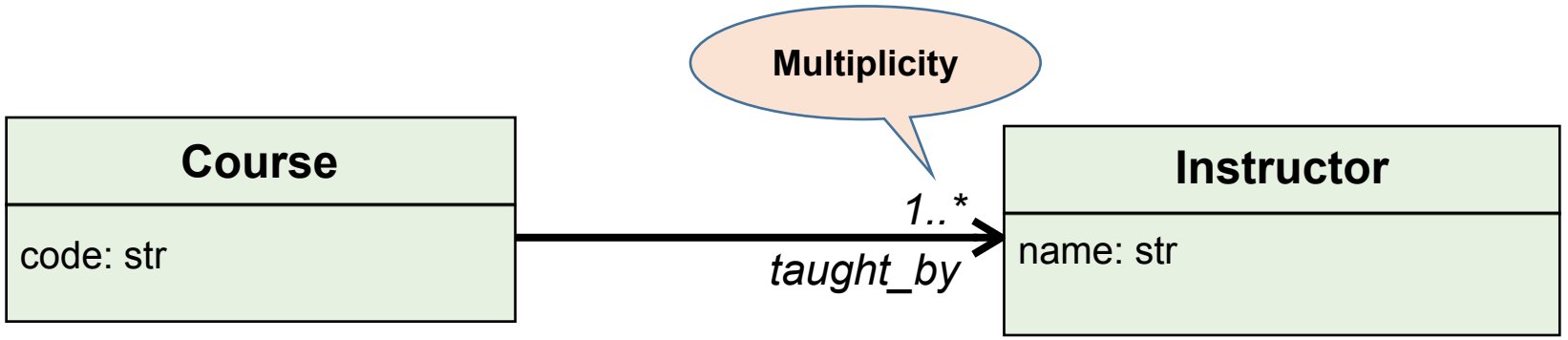
Association Relationships

- In class diagrams, associations represent the connections between classes, e.g. the ways in which their objects may interact:

- ***Multiplicity*** **–** one-to-one, one-to-many, many-to-many, …?
  - E.g. a Doctor has many Patients, but a Patient has one Doctor

- ***Directionality*** **–** is the association uni- or bidirectional?
  - E.g. Lesson → Classroom (one-way), Doctor ↔ Patient (mutual awareness)

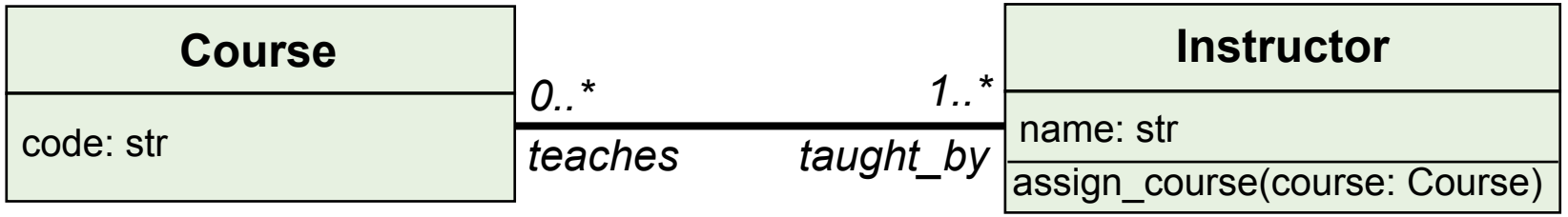
Association Relationships



A Course is taught by one or more Instructors



An Instructor teachers zero or more Courses



(Bi-directional relationship)

Demo!

Aggregation and Composition

- There are two special types of association concerning "whole-part" relationships between objects:

aggregation and composition is a subset of association

- ***Aggregation*** **–** "**x** has a **y**", but **x** and **y** can exist independently
  - *E.g. "a Person has a Pet"*

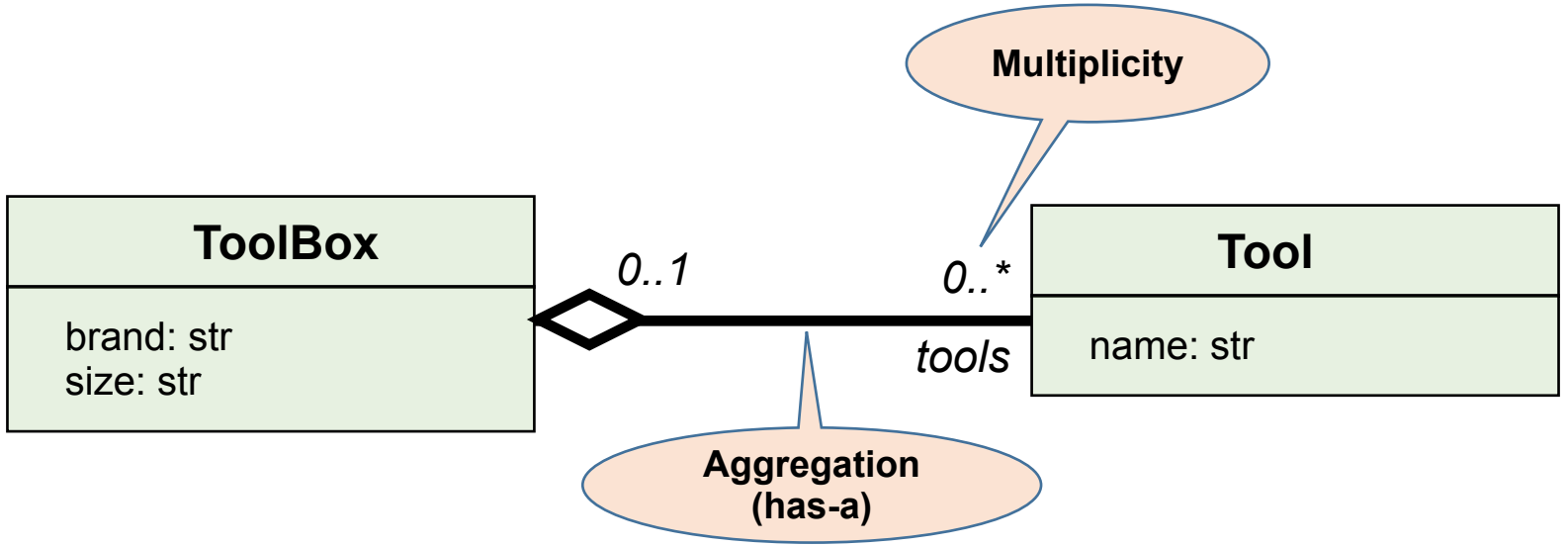
The Person and the Pet can exist independently

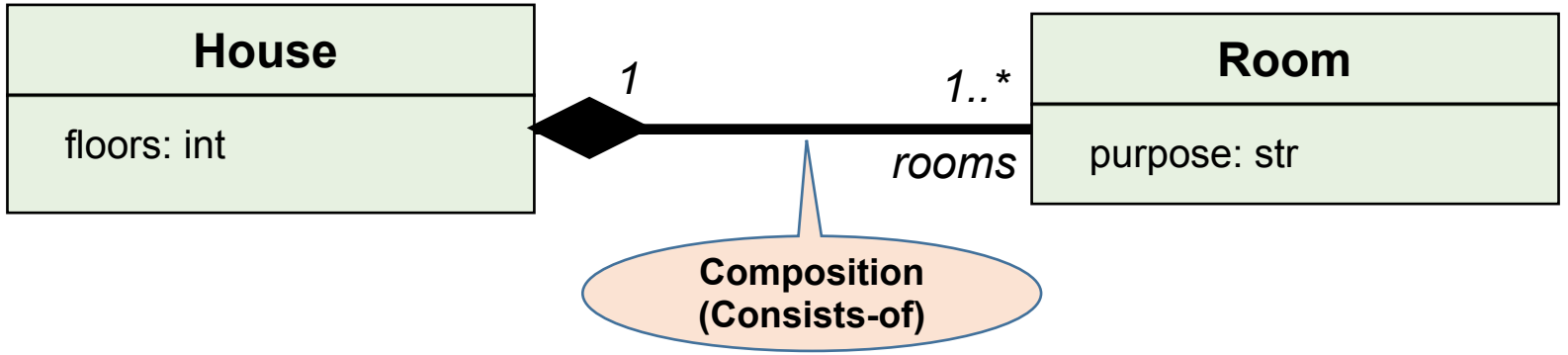
- ***Composition*** **–** "**x** consists of **y**"; **y** cannot exist independently
  - *E.g. "the House consists of Rooms"*

The Room cannot exist without the House

Aggregation and Composition

Demo!





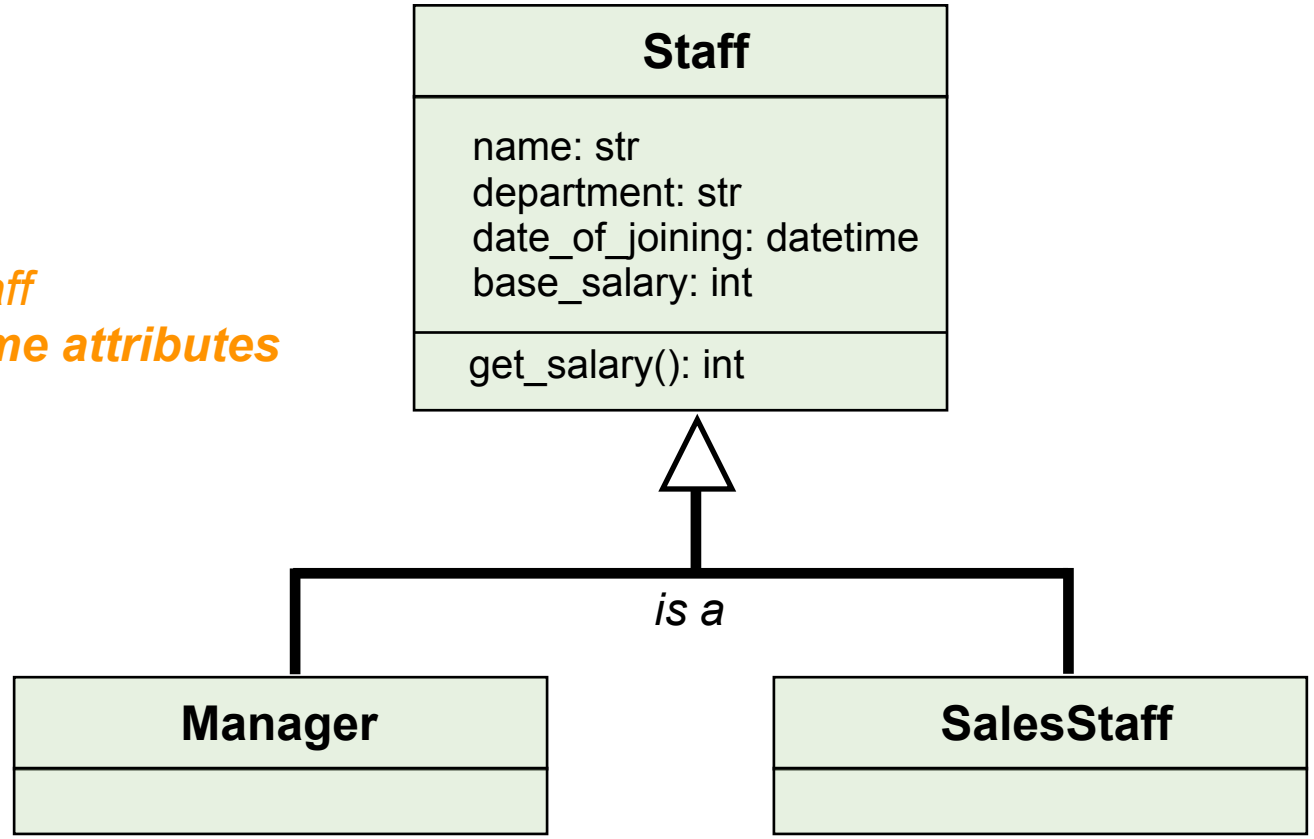
Inheritance

- Inheritance is a concept that allows a new class (subclass) to derive attributes/methods from an existing class (superclass)
  - ***"is-a"*** *relationships, e.g. "a Doctor is a Person"*
  - *Allows for code reuse and the creation of hierarchical relationships*

*Manager and SalesStaff objects 'inherit' the **same attributes** and **methods** as Staff*

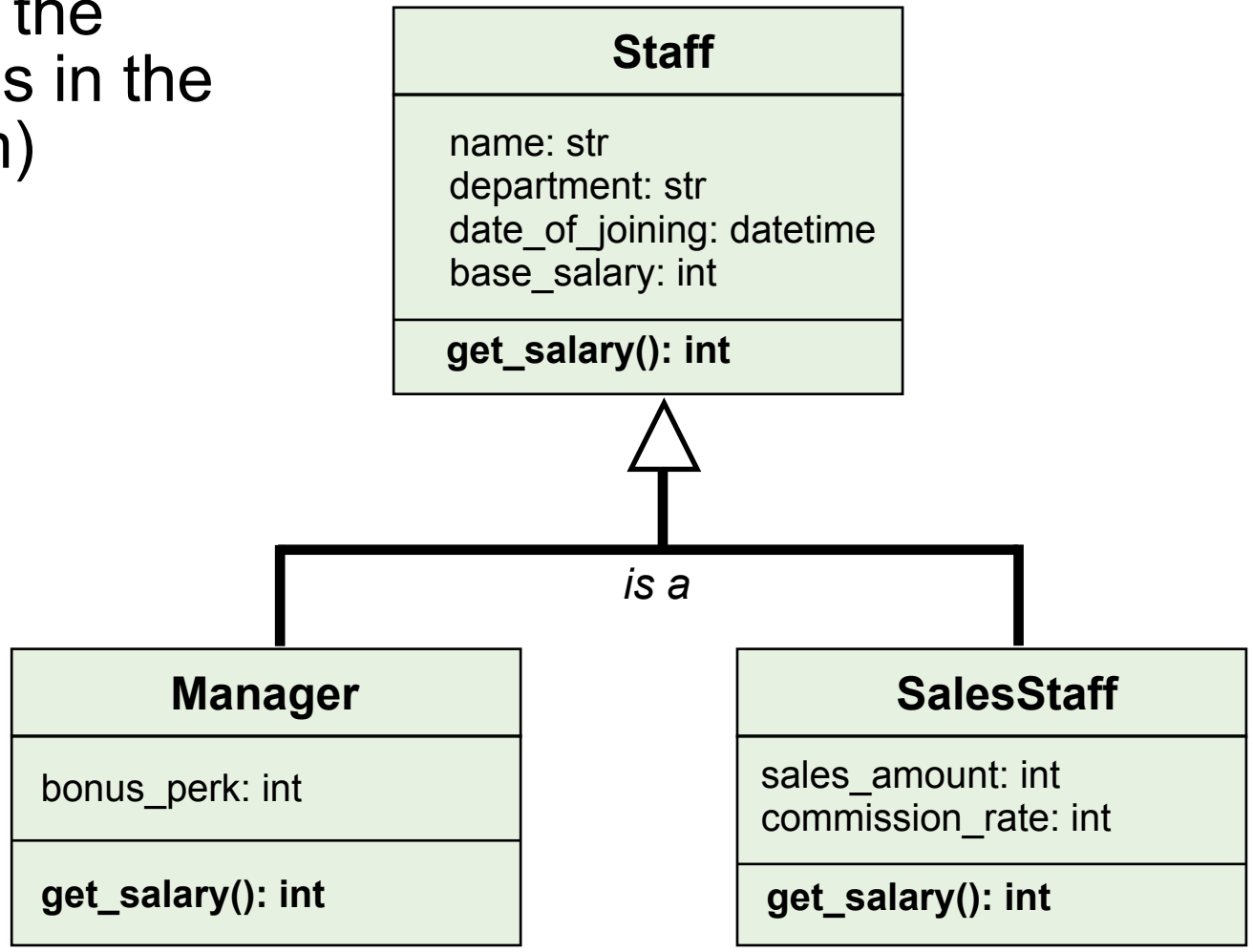
==> code re-use!

Demo!



Polymorphism

- Polymorphism allows for many forms of the same method, depending on the type of the object (i.e. which class in the hierarchy it was instantiated from)



*Three definitions of **get_salary()***

Python automatically executes the correct one for the object's specific type

Demo!

Putting it all Together

Customer Briefing —> C4 Models —> Code

Example Briefing (partial) – GP Surgery System

…

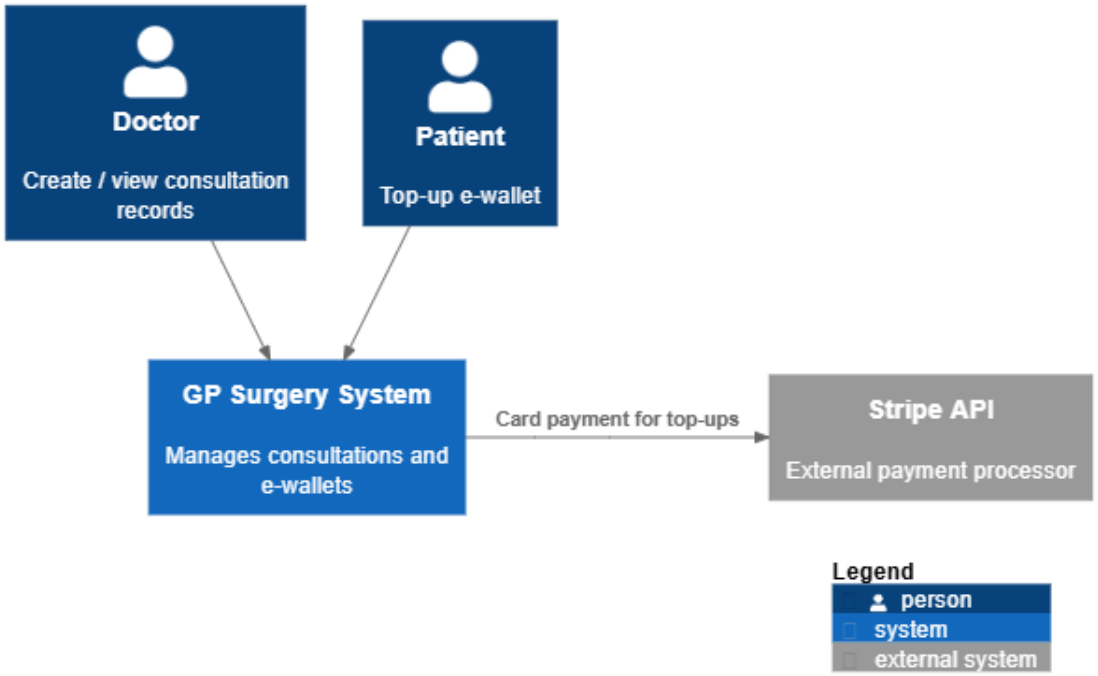
All charges will be handled electronically. Each patient will have an "e-wallet" that they can top up and use to spend on consultations.

…

During a consultation, the doctor enters details such as the diagnosis and prescription. At the end of the consultation, the details are submitted, and the patient's e-wallet is deducted according the consultation length and the doctor's hourly rate.

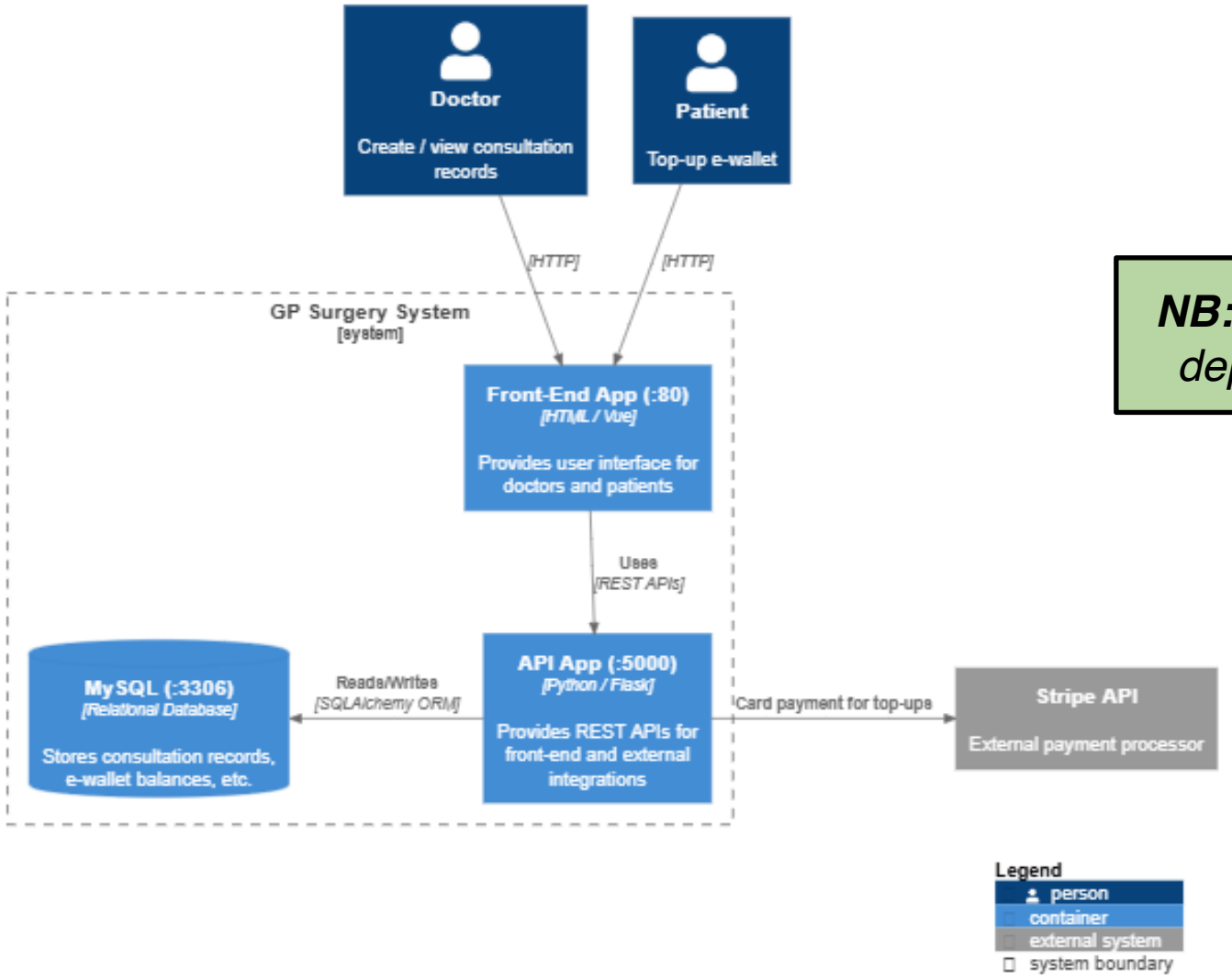
…

Initial C1 (Context) Diagram



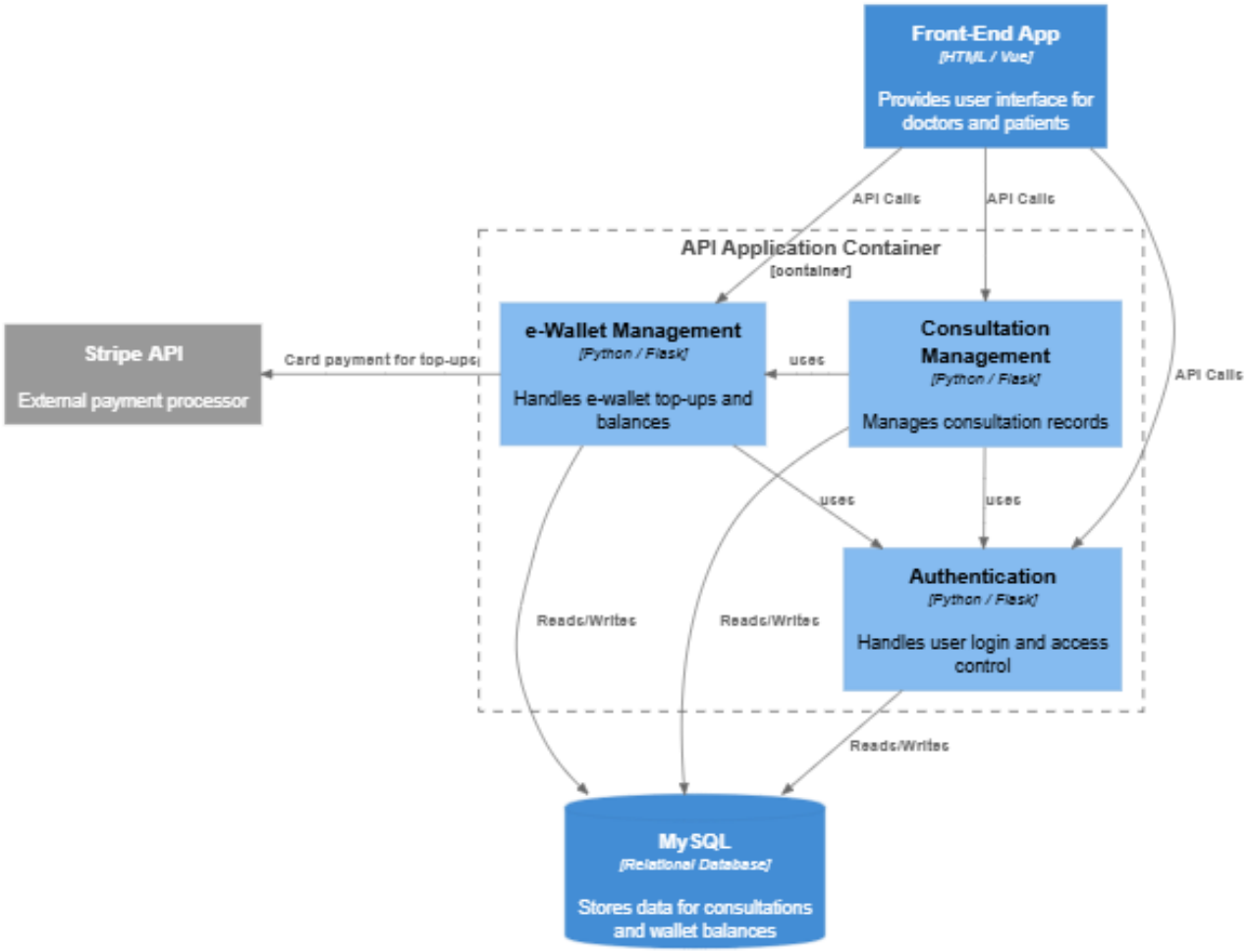
NB: *context diagram likely to "evolve" later as more of the brief is studied*

Initial C2 (Container) Diagram



***NB:** containers can be deployed separately*

Initial C3 (Component) Diagram



NB: *components are* ***NOT*** *separately deployable*

Example Briefing (partial) – GP Surgery System

…

All charges will be handled electronically. Each **patient** will have an "**e-wallet**" that they can top up and use to spend on consultations.

…

During a **consultation**, the **doctor** enters details such as the **diagnosis** and **prescription**. At the end of the consultation, the details are submitted, and the patient's e-wallet is deducted according the consultation **length** and the doctor's **hourly rate**.

…

Bold noun **=** *candidate "domain class" or attribute*

As a User Story?

As a **doctor**, I want to create a **consultation record** so that the **patient's diagnosis** / **prescription** are captured and **charges** are deducted

Ensure that the doctor can:

- Select a registered patient by name
- Select the consulting doctor
- Enter the diagnosis and prescription
- Enter the appointment length in minutes
- Create a consultation record and deduct charges
- Only create a record if patient's e-wallet balance is sufficient

***Pink** = better as an attribute?*

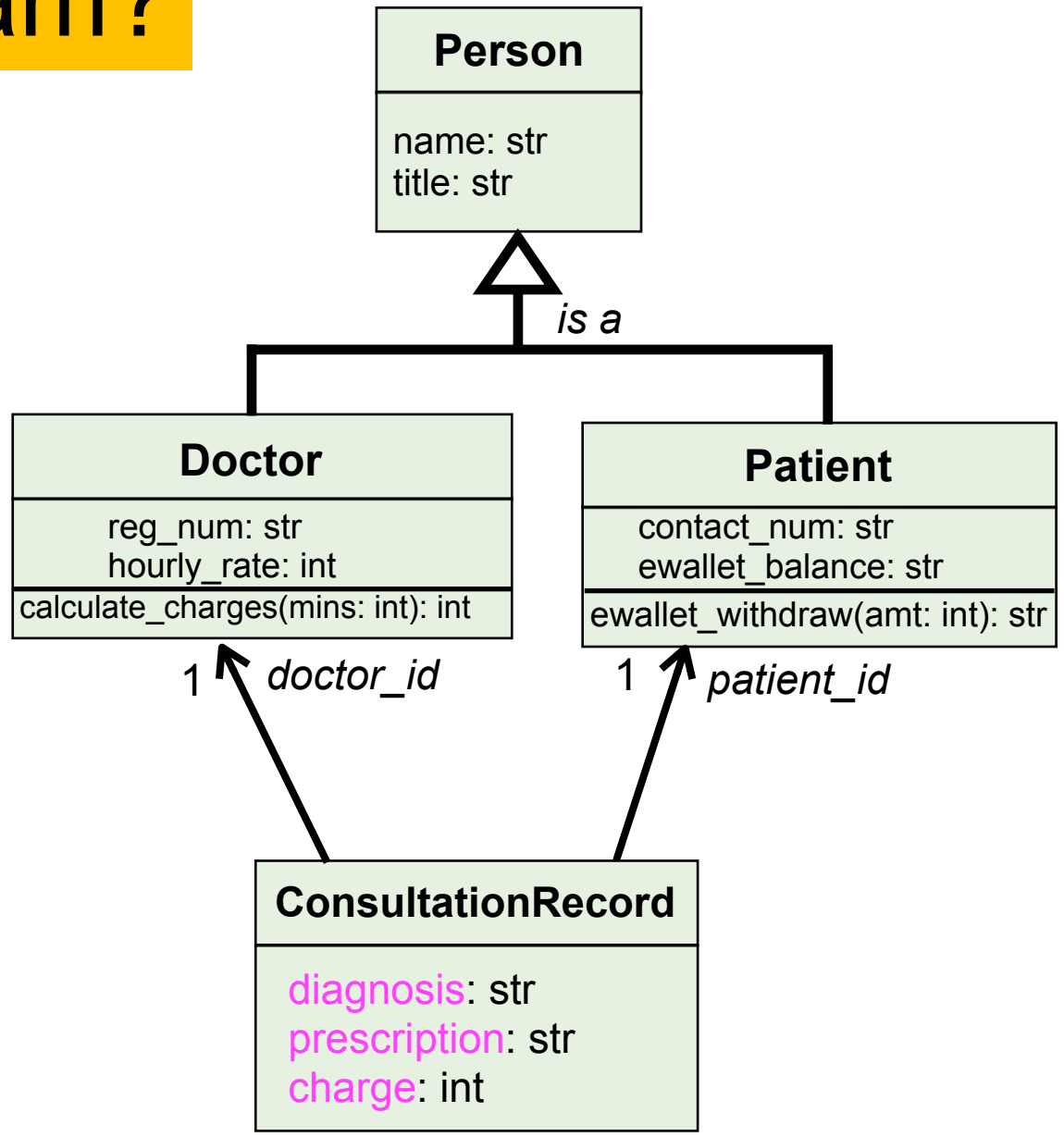
Possible Class Diagram?

(i.e. within the Consultation Management Component)

Demo!

NB: if using Flask-SQLAlchemy, *"data access objects"* are NOT required (it uses the ORM instead)

If using PHP, you may need them (e.g. a PersonDAO class)



Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet ☺
- **http://smu.sg/is212-reflections-2025**



Exercise

- Try sketching the **C1 (Context)** and **C2 (Container)** diagrams for your IS212 project
- Try **modelling any interesting class relationships** implied by this year's IS212 Customer Briefing
- *(Feel free to use our Python/Flask application as a template)*